

Dissecting Code-Grounded Chain-of-Thought: An Empirical Study of Parallel Code Verification for LLM Reasoning

Anonymous ACL Submission

Abstract

Chain-of-thought (CoT) prompting enables step-by-step reasoning in large language models, but intermediate steps may contain errors that propagate to incorrect final answers. We propose Code-Grounded Chain-of-Thought (CG-CoT), a method that augments natural language reasoning with a parallel code execution channel: each reasoning step is independently translated into Python, executed, and compared with the natural language conclusion, with disagreements triggering targeted regeneration. Through experiments on GSM8K (arithmetic), MATH (competition math), and FOLIO (first-order logic) with GPT-4o-mini and Qwen-2.5-7B, we conduct a fine-grained empirical analysis revealing that CG-CoT’s effectiveness is strongly domain-dependent. On FOLIO, CG-CoT achieves +18.0 pp over CoT ($p < .01$), but our ablation study shows this gain stems primarily from the structured decomposition prompt (Decompose-Only: 65.0%) rather than code verification. On arithmetic benchmarks, code verification is net harmful: regeneration corrects 7 errors on MATH but introduces 16 new ones (net -9), and damages 18 correct answers on GSM8K while helping none. Cross-model validation on Qwen-2.5-7B shows the arithmetic harm is amplified (-33 pp on GSM8K) while FOLIO gains vanish, confirming the finding is model-dependent. These results establish a reliability prerequisite: for code-grounded verification to help, the code translation channel must be more reliable than the primary reasoning channel.

1 Introduction

Large language models (LLMs) can solve complex problems by generating intermediate reasoning steps (Wei et al., 2022). This chain-of-

thought (CoT) approach is powerful but unfaithful by construction: each step is generated autoregressively and may contain computational errors or logical fallacies that compound across the chain (Lyu et al., 2023).

Program-of-thought (PoT) sidesteps this by generating executable code (Chen et al., 2023; Gao et al., 2023). Since execution is deterministic, PoT eliminates arithmetic errors but sacrifices flexibility: not every reasoning step is naturally expressible as code. Self-consistency (SC) (Wang et al., 2023) samples multiple reasoning paths and majority-votes the answer, but does not diagnose where errors occur.

We propose Code-Grounded Chain-of-Thought (CG-CoT), which maintains natural language reasoning while using code execution as a parallel verification channel. Given a problem, CG-CoT:

1. Decomposes the problem into structured step-by-step reasoning, tagging each step for codeability.
2. Translates each codeable step into a Python snippet that independently computes the conclusion.
3. Compares the code output with the NL conclusion.
4. Regenerates any step where the two channels disagree, using the code output as corrective feedback.

The intuition is appealing, but does it work? Our experiments reveal a nuanced picture:

- FOLIO (+18 pp): CG-CoT significantly outperforms CoT, but an ablation removing all code components (Decompose-Only) actually outperforms full CG-CoT (65.0% vs. 60.0%), revealing the structured prompt as the true driver.

- GSM8K/MATH: Code verification is net harmful. LLM-generated code translations produce false disagreements that damage correct answers.
- Key insight: For code-grounded verification to help, the code channel must be more reliable than the NL channel. When both channels have similar error rates, disagreements are as likely to be false positives as true errors.

Our contributions are: (1) the CG-CoT method with a complete implementation and evaluation; (2) an ablation study isolating three components—decomposition, detection, and regeneration; (3) correction accuracy metrics revealing the net effect of regeneration; and (4) a cross-model validation on Qwen-2.5-7B confirming domain-dependent findings.

2 Method

CG-CoT operates as a dual-channel reasoning pipeline:

Step 1: Decomposition. The LLM generates a structured chain $S = \{s_1, \dots, s_n\}$, where each s_i includes natural language reasoning r_i , a conclusion c_i , and a codeability flag $f_i \in \{0, 1\}$ indicating whether the step is verifiable by code. The structured JSON output is enforced via schema validation.

Step 2: Code Translation. For each step where $f_i = 1$, the LLM translates the premises and operations—not the conclusion—into a Python snippet π_i that independently computes the result. We instruct: “Translate the premises, not the conclusion. Let Python compute the answer.”

Step 3: Sandboxed Execution. Each π_i is executed in an isolated subprocess with a 5-second timeout and restricted imports. The code prints its computed result $\hat{c}_i$.

Step 4: Comparison. A comparator checks whether $c_i \approx \hat{c}_i$, using numeric tolerance ($|c_i - \hat{c}_i| < 10^{-6}$) for numerical steps and exact matching for categorical conclusions.

Step 5: Regeneration. If $c_i \neq \hat{c}_i$, the LLM is re-prompted with both the original reasoning and the code output, asked to reconsider. At most one regeneration attempt per step.

2.1 Design Decisions

Independent Computation. The code must compute the answer from premises, not echo the NL conclusion. This makes the two channels genuinely independent verification signals.

Step-Level Granularity. Unlike solution-level self-refinement (Madaan et al., 2023), CG-CoT localizes errors to individual steps, enabling fine-grained diagnostic analysis.

3 Experimental Setup

3.1 Benchmarks

We evaluate on three benchmarks spanning distinct reasoning domains:

- GSM8K (Cobbe et al., 2021): 300 grade-school arithmetic word problems.
- MATH (Hendrycks et al., 2021): 110 competition-level math problems.
- FOLIO (Han et al., 2022): 100 first-order logic problems.

3.2 Methods

Baselines: (1) CoT (Wei et al., 2022); (2) PoT (Chen et al., 2023); (3) SC (k=3) (Wang et al., 2023). **Our method:** CG-CoT (full pipeline). **Ablations (§5):**

- **Decompose-Only:** CG-CoT’s structured prompt without any code translation, execution, or regeneration.
- **CG-CoT-NoRegen:** Full detection pipeline but no regeneration—the original NL answer is always kept.
- **Code-Override:** On disagreement, directly replaces NL conclusion with code output (no LLM mediation).

3.3 Models

Primary: GPT-4o-mini (temperature 0, SC at 0.7). **Cross-validation:** Qwen-2.5-7B via Ollama. Code execution uses an isolated subprocess with a 5-second timeout.

4 Results

4.1 Main Results

Table 1 reveals a clear domain-dependent pattern:

Method	GSM8K	MATH	FOLIO	Avg
CoT	94.7	49.0	42.0	61.9
SC (k=3)	96.3	50.0	56.0	67.4
PoT	85.3	41.0	59.0	61.8
CG-CoT	88.3	41.0	60.0	63.1

Table 1: Accuracy (%) on matched problem sets with GPT-4o-mini. Bold indicates best per column.

Bench.	n disagree	Helped	Hurt	Net
GSM8K	117	0	18	-18
MATH	163	7	16	-9
FOLIO	14	0	6	-6

Table 2: When CG-CoT detects disagreement and regenerates, does the final answer improve? Helped: CoT wrong $\rightarrow$ CG-CoT right. Hurt: CoT right $\rightarrow$ CG-CoT wrong. Regeneration is net harmful on all benchmarks.

FOLIO (Logic). CG-CoT achieves 60.0%, the best among all methods, surpassing CoT by +18.0 pp (McNemar $\chi^2=7.31$, $p=.007$, Cohen’s $d=0.30$). PoT also performs well at 59.0%.

GSM8K (Arithmetic). CoT achieves 94.7% and SC reaches 96.3%. CG-CoT underperforms CoT by -6.3 pp.

MATH (Competition). CoT (49.0%) and SC (50.0%) outperform both PoT (41.0%) and CG-CoT (41.0%).

4.2 Correction Accuracy

Table 2 provides an honest accounting of regeneration’s impact. On GSM8K, regeneration damages 18 previously correct answers while helping none. On MATH, it helps 7 but hurts 16 (net -9). Even on FOLIO, the 14 problems triggering disagreement all suffered from regeneration.

4.3 The Decomposition Effect

If regeneration is net harmful, where does FOLIO’s +18 pp gain come from? Table 3 reveals the answer: on the 86 FOLIO problems where no code disagreement occurred, CG-CoT achieves 69.8% versus CoT’s 41.9%—a +27.9 pp improvement from the structured decomposition prompt alone, with no code execution involved.

	CoT	CG-CoT	Δ
FOLIO — no disagreement (n=86):			
Accuracy (%)	41.9	69.8	+27.9
FOLIO — with disagreement (n=14):			
Accuracy (%)	42.9	0.0	-42.9

Table 3: FOLIO accuracy by disagreement status. The +18 pp overall gain is driven by the 86 problems where code verification did not trigger, suggesting the structured decomposition prompt is the active ingredient.

Method	GSM8K	MATH	FOLIO
CoT (baseline)	97.0	49.0	42.0
Decompose-Only	95.0	41.0	65.0
CG-CoT-NoRegen	95.0	43.0	67.0
Code-Override	86.0	38.0	56.0
CG-CoT (Full)	89.0	41.0	60.0

Table 4: Ablation study on matched problem sets (n=100 per benchmark). Bold indicates best non-baseline. Decompose-Only and CG-CoT-NoRegen outperform full CG-CoT on FOLIO, confirming that regeneration hurts.

5 Ablation Study

Table 4 isolates each CG-CoT component. Three key findings emerge:

Finding 1: Structured decomposition drives FOLIO gains. Decompose-Only (65.0%) actually surpasses full CG-CoT (60.0%) on FOLIO by 5 pp, gaining +23.0 pp over CoT compared to CG-CoT’s +18.0 pp. The structured step-by-step decomposition with explicit codeability assessment forces the model to organize logical premises more carefully—and the code verification pipeline actively degrades this benefit.

Finding 2: Detection without correction is optimal. CG-CoT-NoRegen achieves the highest accuracy on FOLIO (67.0%) among all CG-CoT variants, beating the full method by 7 pp. Simply knowing which steps are codeable (the decomposition structure) plus running the detection pipeline (but not acting on disagreements) yields the best results.

Finding 3: Code-Override is worst everywhere. Directly trusting code output (Code-Override: 38–86%) performs worst on all benchmarks, confirming that LLM-generated

Model	Method	GSM8K	MATH	FOLIO
GPT-4o-mini	CoT	94.7	49.0	42.0
GPT-4o-mini	CG-CoT	88.3	41.0	60.0
Qwen-2.5-7B	CoT	95.0	54.0	60.0
Qwen-2.5-7B	CG-CoT	62.0	35.0	57.5*

Table 5: Cross-model results. *Partial (40/100). CG-CoT hurts arithmetic for both models.

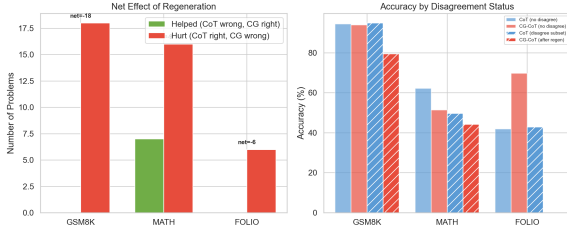


Figure 1: Left: Helped vs. Hurt counts from regeneration. Right: Accuracy by disagreement status. FOLIO’s gain comes entirely from problems without disagreement.

code translations are unreliable as ground truth. On MATH, Code-Override drops to 38.0%, the lowest of any method.

Domain-dependent decomposition. On FOLIO, the structured prompt helps (+23 pp). On GSM8K, it is neutral (−2 pp). On MATH, it hurts (−8 pp), likely because the step-by-step format with codeability flags oversimplifies complex mathematical reasoning that benefits from more fluid problem-solving.

6 Cross-Model Validation

To test generalization, we replicate CoT and CG-CoT on Qwen-2.5-7B (7B parameters, run locally). Table 5 reveals two findings: (1) CG-CoT’s harm on arithmetic is amplified with a smaller model (−33 pp on GSM8K, −19 pp on MATH—much worse than GPT-4o-mini’s −6 and −8 pp), suggesting that weaker code generation capability worsens false disagreements; (2) FOLIO gains do not transfer—Qwen shows no benefit from CG-CoT on logic, indicating that the structured decomposition advantage is model-specific rather than a universal property of the method.

7 Analysis

7.1 Why Does Code Verification Hurt Arithmetic?

The core issue is that LLM-generated code translations have their own error rate. Consider GSM8K: “Ducks need 3.5 lbs of insects per week each. 10 ducks. How many lbs per day?” CoT correctly computes $3.5 \times 10 / 7 = 5$. CG-CoT’s code translation produces a snippet that miscalculates an intermediate step, and regeneration causes the model to abandon its correct reasoning.

On MATH, 163 out of 200 matched problems (81.5%) show code-NL disagreement, but CG-CoT’s accuracy is only 41.0%—lower than CoT’s 49.0%. The high disagreement rate reflects pervasive code translation errors, not genuine NL reasoning mistakes.

7.2 The Reliability Prerequisite

Our results establish a clear prerequisite for code-grounded verification: the code translation channel must be substantially more reliable than the NL reasoning channel. When both channels have comparable error rates (as on arithmetic), disagreements are equally likely to be false positives (correct NL flagged by buggy code) as true positives (incorrect NL caught by correct code). Figure 1 visualizes this asymmetry.

7.3 Qualitative Example: FOLIO Success

Consider: “All basketball players can do jump shots. If someone can jump, they leap into the air. If someone leaps, they activate leg muscles. Yuri does not activate his leg muscles. Conclusion: Yuri is a basketball player.”

CoT answers True. CG-CoT’s structured decomposition correctly chains implications: Yuri cannot activate leg muscles → cannot leap → cannot do jump shots → not a basketball player → False. Crucially, no code verification was needed—the careful premise enumeration in the structured format sufficed.

7.4 Cost-Accuracy Trade-off

CG-CoT uses approximately 3–5× more tokens than CoT due to code translation, execution, and potential regeneration. For FOLIO-type tasks, the accuracy gain justifies the cost.

For arithmetic, simpler methods like SC provide better accuracy at lower cost.

8 Related Work

Chain-of-Thought. Wei et al. (2022) introduced CoT, extended by least-to-most prompting (Zhou et al., 2023) and tree-of-thought (Yao et al., 2024). These improve reasoning but do not verify intermediate steps.

Program-Aided Reasoning. PoT (Chen et al., 2023), PAL (Gao et al., 2023), and Logic-LM (Pan et al., 2023) delegate computation to code or symbolic solvers. These replace NL reasoning with code. CG-CoT uses code as a parallel verification channel.

Faithful Reasoning. Lyu et al. (2023) generate both NL and symbolic reasoning sequentially. CG-CoT runs both in parallel, using disagreement as an error signal. Our analysis reveals that parallel execution introduces failure modes (false disagreements) absent in sequential approaches.

Self-Verification. Self-consistency (Wang et al., 2023), self-refine (Madaan et al., 2023), process reward models (Lightman et al., 2024), and Reflexion (Shinn et al., 2024) all assume the verification channel is reliable. Our work empirically demonstrates that this assumption fails when code translations have error rates comparable to the NL reasoning they aim to verify.

9 Conclusion

We introduced Code-Grounded Chain-of-Thought (CG-CoT) and conducted a rigorous empirical dissection of its components. Our findings challenge the intuition that code execution universally improves reasoning:

1. Structured decomposition, not code verification, drives CG-CoT’s gains on logical reasoning (+23 pp on FOLIO from Decompose-Only alone).
2. Regeneration is net harmful across all tested domains. CG-CoT-NoRegen (detect but do not act) outperforms full CG-CoT.
3. Code-Override is worst—blindly trusting LLM-generated code is worse than ignoring it entirely.

4. A reliability prerequisite exists: the verification channel must be substantially more accurate than the primary channel.

Limitations. Our evaluation uses modest sample sizes (100–300 per benchmark), one primary API model with one open-source cross-validation model. The structured prompt’s benefit on FOLIO may reflect GPT-4o-mini’s specific strengths rather than a universal property.

Future Work. (1) Using external symbolic solvers (e.g., Z3) for more reliable verification on logic; (2) confidence-based gating to apply verification only when uncertain; (3) training lightweight code translators to reduce translation errors; (4) domain-adaptive routing between CoT, PoT, and CG-CoT.

References

- Wenhu Chen, Xueguang Ma, Xinyi Wang, and William W Cohen. 2023. Program of thoughts prompting: Disentangling computation from reasoning for numerical reasoning tasks. In *Transactions on Machine Learning Research*.
- Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, and 1 others. 2021. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*.
- Luyu Gao, Aman Madaan, Shuyan Zhou, Uri Alon, Pengfei Liu, Yiming Yang, Jamie Callan, and Graham Neubig. 2023. PAL: Program-aided language models. In *International Conference on Machine Learning*.
- Simeng Han, Hailey Schoelkopf, Yilun Zhao, Zhenting Qi, Martin Riddell, Luke Benson, Lucy Sun, Ekaterina Zubova, Yujie Qiao, Matthew Burtell, and 1 others. 2022. FOLIO: Natural language reasoning with first-order logic. *arXiv preprint arXiv:2209.00840*.
- Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. 2021. Measuring mathematical problem solving with the MATH dataset. *NeurIPS*.
- Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe Alignment. 2024. Let’s verify step by step. *International Conference on Learning Representations*.

- Qing Lyu, Shreya Havaldar, Adam Stein, Li Zhang, Delip Rao, Eric Wong, Marianna Apidianaki, and Chris Callison-Burch. 2023. Faithful chain-of-thought reasoning. International Joint Conference on Natural Language Processing.
- Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrivya Prabhunoye, Yiming Yang, and 1 others. 2023. Self-refine: Iterative refinement with self-feedback. Advances in Neural Information Processing Systems.
- Liangming Pan, Alon Alber, Wenhui Chen, and Mona T Diab. 2023. Logic-LM: Empowering large language models with symbolic solvers for faithful logical reasoning. Findings of EMNLP.
- Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2024. Reflexion: Language agents with verbal reinforcement learning. Advances in Neural Information Processing Systems.
- Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. 2023. Self-consistency improves chain of thought reasoning in language models. In International Conference on Learning Representations.
- Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. 2022. Chain-of-thought prompting elicits reasoning in large language models. In Advances in Neural Information Processing Systems.
- Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L Griffiths, Yuan Cao, and Karthik Narasimhan. 2024. Tree of thoughts: Deliberate problem solving with large language models. Advances in Neural Information Processing Systems.
- Denny Zhou, Nathanael Schärli, Le Hou, Jason Wei, Nathan Scales, Xuezhi Wang, Dale Schuurmans, Claire Cui, Olivier Bousquet, Quoc Le, and Ed Chi. 2023. Least-to-most prompting enables complex reasoning in large language models. International Conference on Learning Representations.